LegoMap: Optimization for High-Throughput Transformer Computing on AI Engine-Based FPGAs

***Abstract*—Transformer layers dominate the computational workload of transformer-based LLMs, creating a strong need for efficient transformer layer acceleration. The AI engine-based FPGA architecture offers the potential for significantly higher efficiency for transformer layer computing compared to conventional FPGAs. In this work, we present an integer linear programming-based approach, called LegoMap, which automatically maps the transformer layer onto AI engine-based FPGAs. Our method jointly optimizes AI engine core allocation, memory usage, and scheduling. Evaluated on the Versal ACAP, it delivers about $30\% - 58\%$ throughput improvement over the state-of-the-art automatic approach across different workloads.**

***Index Terms*—Transformer, Hardware Accelerator, AI Engines, Versal ACAP, Mapping Framework, Scheduling**

I. Introduction

Transformer models [1] have become extremely popular in recent years, with wide adoption in natural language processing [2], [3] and computer vision [4]. With their growth in model size and application domains, executing transformer models requires substantial hardware resources, resulting in significant power consumption and increased execution time. Within each transformer-based model, the transformer layer repeats many times and constitutes the majority of the workload, making it computationally intensive and memory-demanding.

Modern heterogeneous FPGAs, such as the AMD/Xilinx Versal Adaptive Compute Acceleration Platform (ACAP) [5], have emerged as a promising platform for accelerating transformer computing. Versal ACAP integrates the programmable logic (PL) with arrays of specialized AI Engines (AIEs), offering both architectural flexibility and high compute density. As listed in Figure 1, the Versal ACAP architecture can be viewed in three levels: (1) The AIE cores, each with local memory, provide high-throughput Single Instruction Multiple Data (SIMD) processing for dense numerical workloads; (2) the PL offers on-chip buffers for intermediate data management, and reconfigurable logic for customized functions; and (3) the external Dynamic Random Access Memory (DRAM) and integrated ARM cores provide large capacity storage and system-level control. In addition, Versal ACAP includes a Processor System (PS) for execution orchestration and host interaction, a Network-on-Chip (NoC) for on-chip data movement, and PLIO streaming interfaces between the PL and AIE array for high-bandwidth transfers. This hierarchical design enables a balance between performance and reconfigurability, making Versal ACAP a suitable platform for accelerating transformer layers.

Several research efforts have attempted to leverage ACAP for machine learning acceleration. CHARM [6], [7] proposed a deep learning inference accelerator and an automatic framework to map general matrix multiplications (GEMMs) onto the ACAP architecture. It provided detailed analyses and modeling of the computation and data movement. AutoMM [8] extended CHARM and introduced an automatic framework that systematically generates the accelerator designs for GEMM. However, both works primarily focused on optimizing individual GEMM operators, without exploration of flexible resource allocation and inter-operator scheduling. MaxEVA [9] further explored efficient mapping strategies for AIEs, generating high-performance GEMM acceleration designs that exploit AIE architecture features. It achieves high utilization when matrix dimensions align with the design's native configuration. However, transformer layers involve GEMMs of varying operand dimensions, which makes it difficult for fixed-dimension designs to consistently achieve peak performance. RSN [10] introduced a novel ISA abstraction tailored for deep neural networks, enabling more flexible configurations and achieving highly effective hardware solutions. However, RSN is not an automatic framework and still relies on manual analysis and sophisticated design choices. As a result, while RSN shows the potential of flexible architectures, the need for systematic automation to explore the large design space of transformer layers still remains. EQ-ViT [11] and SSR [12] are focused on architectural level optimizations and do not cover loop tiling, which is a key component of workload mapping.

Overall, most prior works either rely on manual design techniques or target conventional FPGA architectures without AI engines. In addition, these methods typically focus on isolated matrix multiplications and ignore inter-operator interactions such as operator fusion. The most relevant existing approach is CHARM [6], [7], which is likely the only prior work on automatic workload mapping for AI-engine–based FPGAs. However, CHARM does not account for operator fusion and employs a greedy heuristic, which is known to struggle with producing high-quality solutions. Moreover, AIE array partitioning is manually performed offline in CHARM.

Hardware resource allocation. A fundamental challenge is how to properly allocate hardware resources within and across operators. At the intra-operator level, tiling factors at the DRAM, PL buffer, and AIE array levels directly affect both performance and resource usage. For instance, large GEMMs mapped onto a small AIE subarray suffer

from low data reuse and excessive data movement, limiting throughput. Conversely, smaller GEMMs mapped onto a large AIE subarray waste computational capacity due to under-utilization. At the inter-operator level, the AIE array itself can be partitioned into multiple subarrays to execute different operators concurrently. While this improves parallelism, it also introduces competition for PLIO channels and buffer capacity. Balancing utilization, data reuse, and communication efficiency across these partitioning choices is therefore highly non-trivial.

Operator scheduling. Beyond resource allocation, the execution order and concurrency of operators introduce another level of complexity. The operators within a transformer layer can be scheduled sequentially or in parallel, each with trade-offs. Sequential scheduling is straightforward and minimizes resource contention, but may lead to poor utilization and long execution time. Parallel scheduling reduces overall execution time by overlapping operators, but requires careful coordination to satisfy dependencies and avoid resource conflicts. Furthermore, scheduling strategies such as concatenating GEMMs across batches or fusing adjacent operators change both the execution order and the resource demand, complicating the search for an efficient global schedule.

Joint optimization complexity. Resource allocation and operator scheduling are tightly coupled. Decisions about tiling and subarray allocation directly affect scheduling feasibility, while scheduling choices determine how resources are utilized over time. Optimizing these dimensions together creates a huge and highly constrained design space, which is impractical to explore manually.

To address these challenges, we propose an automatic framework, called **LegoMap**, for mapping transformer layers on AI engine-based FPGA accelerators. Our key contributions are as follows:

- **Automatic mapping framework.** We design a two-stage framework that first generates candidate mappings for each operator, and then applies integer linear programming (ILP) that couples mapping selection, fusion, and scheduling under shared AIE/PLIO/buffer limits. This global coupling approach empirically outperforms greedy heuristics in identifying high-quality solutions.
- **Inter-operator optimization.** Our approach covers inter-operator optimization through fusion in addition to intra-operator optimization.
- **Automatic AIE array partitioning.** The partitioning of AIE array into operator-specific subarrays is performed automatically in our optimization framework instead of manual offline decisions.
- **ILP solving acceleration.** One drawback of ILP is its slow runtime. To address this issue, we propose two customized techniques that can significantly accelerate the ILP computation without sacrificing solution quality.
- **Comprehensive evaluation.** We evaluate our framework on Versal ACAP using representative transformer layers. Results show an average of 30%-58% throughput improvement over the state-of-the-art automatic method.

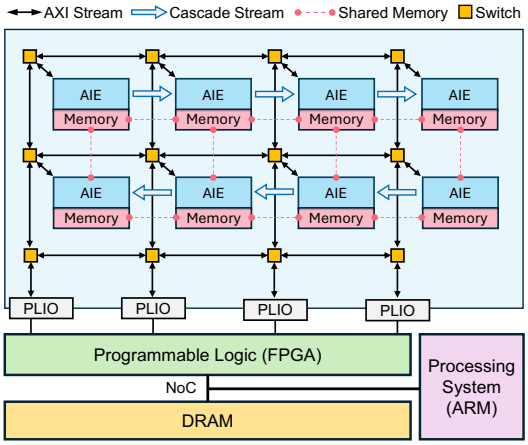

Fig. 1: The AI engine-based FPGA architecture.

II. Background and Motivation

In this section, we first introduce the architecture of the Versal ACAP platform. Then we illustrate how a single matrix multiplication is mapped onto the Versal ACAP. Based on this, we extend the context to the scheduling of a transformer layer.

A. Overview of Versal ACAP Architecture

Figure 1 shows the architecture of the AMD/Xilinx Versal ACAP platform. This heterogeneous platform features an AI Engine (AIE) array, the Programmable Logic (PL), and an ARM Processing System (PS).

The AI Engine array is designed to provide high-performance, energy-efficient computation for dense numerical tasks such as matrix multiplications and convolutions. Taking the Versal Series VC1902 as an example, the AIE array consists of 8×50 AIE cores, each of which contains a 7-way Very Long Instruction Word (VLIW) processor that supports vector operations running up to 1 GHz. Each AIE core also has its own local scratchpad memory. An overview of the interconnect in the AIE array is illustrated in Figure 1. Three types of communication are supported: (1) The local memory in each AIE core is shared with its north, south, and east or west neighbor, depending on the location of the core. (2) One-way cascade stream exists between AI Engine cores in the horizontal direction, providing native support for chaining multiply-accumulate operations. (3) The AXI interconnect provides streaming connections between non-neighboring AI Engine cores and facilitates communication from the PL to the AIE array. The AXI network also supports circuit-switched and packet-switched connections, and both execute deterministically under a compiled design. Circuit-switched communication establishes dedicated routes for high-throughput transfers, while packet-switched communication provides flexible routing via header-directed delivery.

PL represents the reconfigurable Field-Programmable Gate Array (FPGA) fabric, which contains configurable logic blocks (look-up tables and flip flops), digital signal processors (DSPs), and on-chip memory such as Block Random Access Memory (BRAM) and Ultra Random Access Memory

```
1: # DRAM
2: for i3 in range (iDRAM):
3: for j3 in range (jDRAM):
4: for k3 in range (kDRAM):
5:  # On-chip Memory
6:  for i2 in range (iPL):
7:  for j2 in range (jPL):
8:  for k2 in range (kPL):
9:      # Spatial Unroll on AIE Array
10:     for i1 in range (iArr):
11:     for j1 in range (jArr):
12:     for k1 in range (kArr):
13:     # Single AIE SIMD
14:         for i0 in range (iAIE):
15:         for j0 in range (jAIE):
16:         for k0 in range (kAIE):
17:             C[i0][j0]+=A[i0][k0]×B[k0][j0]
```

Fig. 2: Matrix multiplication represented in nested-loops.

(URAM). PL offers architectural flexibility, allowing designers to implement customized logic functions, such as nonlinear operations, reduction operations, and control logic. Communication between PL and the AIE array is through the PLIOs, enabling the PL to feed data to the AIE array and collect results back. PL also has direct access to the off-chip DRAM. The BRAM and URAM in the PL are typically used as buffers to store tiles of input data and intermediate results. Double buffering is often used in the PL to overlap data movement with computation.

The PS comprises general-purpose ARM cores, memory controllers, and peripheral interfaces. It serves as the top-level controller for any applications running on the platform. The PS is directly connected to the network-on-chip and coordinates with DRAM, the PL, and the AIE array.

B. Matrix Multiplication on Versal ACAP

Figure 2 shows a general matrix multiplication in a nested-loop representation. The input matrices are originally stored in the DRAM, and are transferred to the on-chip buffer in the PL. When the matrices are too large to fit entirely in on-chip memory, tiles of the matrices are streamed sequentially, as represented by the outermost loops (i_3, j_3, k_3). These tiles are further partitioned and sent to the AIE array through PLIOs (Lines 6-8). Computation on the AIE array can be spatially unrolled, with groups of AIE cores executing in parallel. Lines 10-12 represent such a kernel mapped to a 2D subarray of AIE cores. Each AIE core uses its local memory to perform a small matrix multiplication, while partial sums are accumulated across the cores in the same row through the dedicated cascade stream. After the AIE computation is finished, results are returned to PL buffers, accumulated if necessary, and finally written back to DRAM.

Several decision elements affect the mapping of General Matrix Multiplication (GEMM) on the Versal platform.

- **Loop tiling**: Tile sizes can be chosen at multiple levels of the hierarchy. The tiling configuration determines memory footprint, number of active AIE cores, and computational parallelism.

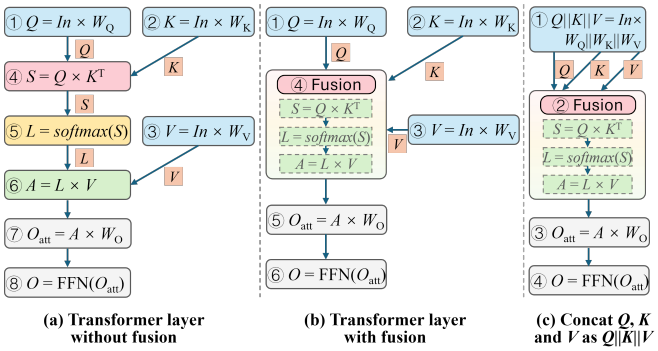

Fig. 3: Transformer computation.

- **Computation ordering**: The traversal order of cores affects data reuse and movement. Favorable ordering reduces redundant transfers and improves overlap between computation and communication, while poor ordering increases memory traffic and execution time.
- **Broadcasting**: When the same data is consumed by multiple AIE cores, it can be transferred once from the PL and broadcast across the array. This eliminates the need to replicate data transfers from PL to each consumer core, thereby reducing the number of PLIOs required.
- **Packet-switched interconnect**: The AIE interconnect allows multiple streams to share the same route through packet switching. This reduces pressure on PL–AIE interfaces by avoiding dedicated PLIOs for every stream.

Different choices of decision elements introduce different GEMM mappings, resulting in varying resource demands and performance. We summarize the key demands and performance metrics as follows: **#AIE** denotes the number of AIE cores required for the tiled matrix multiplication. Please note that a computation kernel is typically mapped to a two-dimensional array of AIEs. Here, we use the number of AIE cores in this array for simplicity. **#PLIO** denotes the number of PLIOs needed to support the data stream between the PL and the AIE array. **Buffer** denotes the size of the on-chip memory required to accommodate the operand matrices. **Execution time** denotes the time elapsed from the start to the end of a task.

C. Transformer Layer Computation on Versal ACAP

Figure 3 (a) illustrates the architecture of a transformer layer. To simplify the visualization, we omit the layer normalization, residual connection, and the element-wise activation functions. The matrix multiplication operators in attention sublayer (①-⑦) and the feedforward sublayer (⑧) account for the majority of both computation and data movement within a transformer layer. In addition, the complexity of the attention sublayer grows quadratically with the sequence length [13]–[16]. The edges (labeled Q, K, V, S, L, and A) denote the matrices transferred between these computation blocks. Since the softmax computation can be overlapped [12], [17]–[19] in a pipeline, overall performance is dominated by both the mapping and scheduling of the matrix multiplication opera-

tions. The matrix multiplications and their dependency can be abstracted into a DAG. Each node in the DAG represents an operator, and each edge denotes a data dependency between operators.

The complexity of mapping an attention layer onto Versal ACAP comes in threefold: (1) **Node level**: Each GEMM operator can be mapped with different configurations, introducing different hardware resource demands and execution time. (2) **Graph level**: On top of the single operator mapping choices, scheduling the operators brings another level of complexity. Operators can be mapped onto the AIE array either sequentially or spatially. In sequential mapping, operators are executed one by one on the same hardware resources, following their topological ordering in the DAG. For spatial mapping, one or more operators can be executed in parallel on separate hardware resources, provided that data dependency is satisfied. (3) **Transformer layer level**: A transformer layer can be represented by different graph structures due to several reasons. Operator fusion is an effective strategy to improve the overall efficiency [17], [20]–[22]. When a tiled matrix is produced by one operator, it can be directly forwarded to the next operator as an operand. This avoids the costly process of writing the tile back to DRAM and fetching it again into on-chip memory. Figure 3(b) shows an example of the corresponding DAG representation with a fusion of operators S, L, and A. In this scenario, nodes ④-⑥ from Figure 3(a) merge into one, changing the DAG from the one in Figure 3(a) to Figure 3(b). Another scenario that changes the graph is matrix concatenation. In Figure 3(b), Operators ①- ③ share a common input operand In, thus the other operand, matrices W_Q, W_K, W_V can be concatenated together. In this case, the corresponding graph changes from Figure 3(b) to Figure 3(c).

In summary, mapping transformer layers on ACAP requires decisions across different levels, resulting in a design space that is both extremely large and highly constrained, making manual exploration infeasible. This motivates the need for an analytical model that can quickly and accurately estimate execution time and resource usage, providing the foundation for systematic mapping decisions.

III. Analytical Models

In this section, we present an analytical model to estimate the execution time and resource demand of a GEMM mapping on the Versal ACAP platform. We adopt CHARM's per-operator model [6] and extend it to support LegoMap's sequential/parallel/fusion decisions by capturing peak hardware usage under concurrency and reduced DRAM traffic under fusion. The models are to guide solution search, and the final performance and utilization are reported from tool reports.

We reuse the notations from the nested-loop representation in Figure 2, the operands' dimensions are defined as: $\mathrm{C}(I \times J) = \mathrm{A}(I \times K) \times \mathrm{B}(K \times J)$.

The hardware resource requirements are summarized as follows. The number of PLIOs depends on the data volume required by the AIE array, which is proportional to the array dimensions used for mapping. Computation-To-Communication ratio (CTC) is defined to capture the effect of circuit-switched and packet-switched interconnects. The number of AIE cores is determined directly by the dimensions of the mapped AIE array. The buffer demand is determined by the tile sizes stored in PL on-chip memory, with a factor of 2 reflecting the double-buffering strategy.

$$\begin{aligned}\#\mathrm{PLIO} = &\lceil \mathrm{i_{AIE}} \cdot \mathrm{k_{AIE}} \cdot \mathrm{i_{Arr}} \cdot \mathrm{k_{Arr}}/CTC \rceil \\ &+ \lceil \mathrm{k_{AIE}} \cdot \mathrm{j_{AIE}} \cdot \mathrm{k_{Arr}} \cdot \mathrm{j_{Arr}}/CTC \rceil \\ &+ \lceil \mathrm{i_{AIE}} \cdot \mathrm{j_{AIE}} \cdot \mathrm{i_{Arr}} \cdot \mathrm{j_{Arr}}/CTC \rceil \end{aligned} \tag{1}$$

$$\mathrm{Buffer} = \mathrm{Buffer_A} + \mathrm{Buffer_B} + \mathrm{Buffer_C} \tag{2}$$

$$\mathrm{Buffer_A} = 2 \cdot (\mathrm{i_{AIE}} \cdot \mathrm{k_{AIE}} \cdot \mathrm{i_{Arr}} \cdot \mathrm{k_{Arr}} \cdot \mathrm{i_{PL}} \cdot \mathrm{k_{PL}}) \tag{3}$$

$$\mathrm{Buffer_B} = 2 \cdot (\mathrm{k_{AIE}} \cdot \mathrm{j_{AIE}} \cdot \mathrm{k_{Arr}} \cdot \mathrm{j_{Arr}} \cdot \mathrm{k_{PL}} \cdot \mathrm{j_{PL}}) \tag{4}$$

$$\mathrm{Buffer_C} = 2 \cdot (\mathrm{i_{AIE}} \cdot \mathrm{j_{AIE}} \cdot \mathrm{i_{Arr}} \cdot \mathrm{j_{Arr}} \cdot \mathrm{i_{PL}} \cdot \mathrm{j_{PL}}) \tag{5}$$

$$\#\mathrm{AIE} = \mathrm{i_{Arr}} \cdot \mathrm{j_{Arr}} \cdot \mathrm{k_{Arr}} \tag{6}$$

Considering the double buffering and the pipeline of data transfer and computation, the execution time is determined by the maximum of three components: communication time $\mathrm{Time_{Load}}$, $\mathrm{Time_{Store}}$, and computation time $\mathrm{Time_{Comp}}$. Computation time is further determined by the computation efficiency of each AIE core (MAC). And the communication time is determined by the data transfer volume and the DRAM bandwidth. Both (Multiply-Accumulate) MAC and DRAM bandwidth are captured from profiling.

$$\mathrm{Time_{Total}} = \max(\mathrm{Time_{Comp}}, \mathrm{Time_{Load}}, \mathrm{Time_{Store}}) \tag{7}$$

$$\mathrm{Time_{Comp}} = \frac{I \times J \times K}{\mathrm{MAC} \cdot \#\mathrm{AIE}} \tag{8}$$

$$\mathrm{Time_{Load}} = \frac{\mathrm{Buffer_A} + \mathrm{Buffer_B}}{\mathrm{Bandwidth_{Load}}} \tag{9}$$

$$\mathrm{Time_{Store}} = \frac{\mathrm{Buffer_C}}{\mathrm{Bandwidth_{Store}}} \tag{10}$$

- Sequential mapping: Operators run one after another on the same AIE array, sharing the same hardware resources. The total execution time is simply the sum of the individual execution times.
- Parallel mapping: Operators run simultaneously on separate AIE arrays. Peak hardware demand is the sum across concurrently active operators, while the total execution time is the maximum execution time among them.
- Fusion: When two operators are fused, the producer's output is forwarded directly to the consumer through on-chip connections, avoiding an intermediate round trip to DRAM. Accordingly, the fused candidate omits the intermediate DRAM load/store terms.

These extensions allow the model to capture the execution of a complete transformer layer, where multiple independent operators may execute sequentially, concurrently, or in fusion.

IV. Optimization

A. Problem Definition

We formulate the mapping of a transformer layer onto the Versal ACAP as a constrained optimization problem.

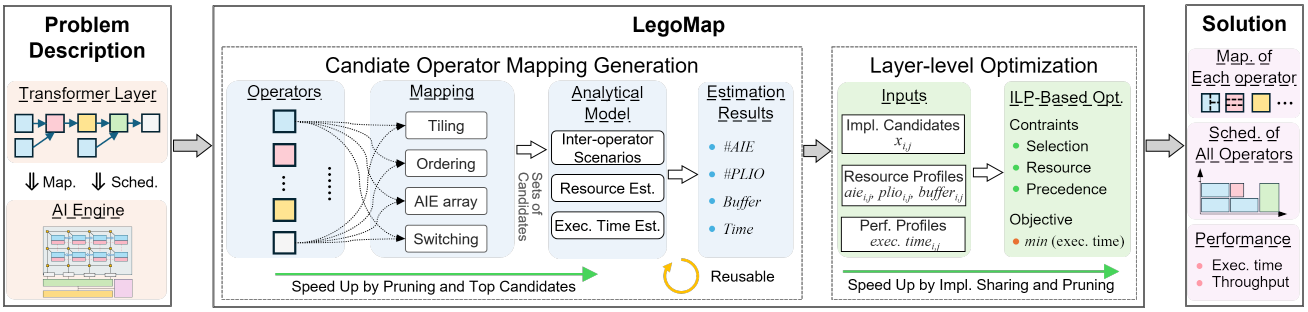

Fig. 4: Overview of the proposed LegoMap methodology.

The input consists of:

- A transformer layer represented as a directed acyclic graph (DAG) $G = (V, E)$, where each node $Op_i \in V$ corresponds to an operator (e.g., GEMM or softmax), and each edge $(Op_i, Op_j) \in E$ denotes a data dependency between operators.
- A target AI engine-based FPGA platform, characterized by its available hardware resources: number of AIE tiles ($\#AIE$), number of programmable logic I/O channels ($\#PLIO$), and on-chip buffer capacity (*Buffer*).

The design decisions are:

- **Operator mapping:** for each operator Op_i, select a mapping from a set of candidates with specific configurations. The configurations involve tiling factors, computation ordering (including both operator ordering and loop tile ordering), broadcasting, and packet-switching factors. Each mapping also comes with an execution time and resource requirement on the number of AIEs, number of PLIOs, and on-chip buffer. Please note that the AIE tile allocation to operators satisfying total AIE number constraint is virtually a partitioning of the given AIE array.
- **Operator scheduling:** assign a start time T_i^{start} and finish time T_i^{finish} to each operator, respecting dependency constraints and allowing concurrent execution of independent operators when hardware resources permit.

The objective is to maximize the overall throughput by minimizing the maximum finish time across all operators in the transformer layer:

$$\min T_{total} = \max_i \{T_i^{finish}\}, \tag{11}$$

subject to the constraints:

- At any point in time, the total usage of AIE tiles, PLIOs, and buffer across all active operators must not exceed the available resources.
- Data dependencies must be satisfied, i.e., $T_j^{start} \geq T_i^{finish}$ for every edge $(Op_i, Op_j) \in E$.

B. Overview of the LegoMap Framework

The proposed framework consists of two hierarchical levels, as illustrated in Figure 4: (1) **Candidate Operator Mapping Generation**, and (2) **Layer-Level Optimization**. At the first level, candidate operator mapping generation automatically produces a finite set of high-quality (non-inferior) solutions for both individual and fused operators. Each candidate specifies loop tiling, loop tile ordering, and hardware resource allocation (e.g., AIE cores). At the second level, layer-level optimization selects among these candidate mappings and composes them into a complete mapping solution for a transformer layer. In addition to candidate selection, this level determines operator fusion decisions and scheduling across operators.

This approach not only applies a divide-and-conquer strategy to manage the high complexity of mapping an entire layer with numerous decision variables, but it also enables design reuse. The candidate operator mappings can be reused across different transformer layers and even across different transformer models. As a simple analogy, the candidate operator mappings serve as the Lego pieces, while the layer-level optimization assembles them into the final Lego toy. The same set of Lego pieces can be used to build different toys. This is the key reason we name our method **LegoMap**.

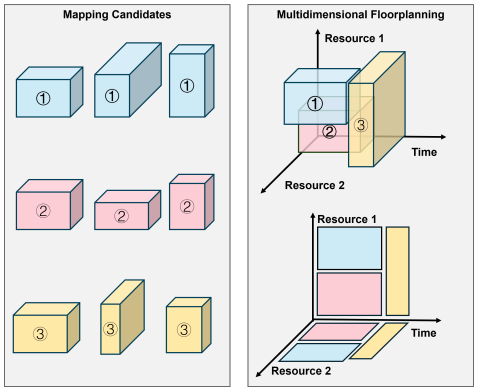

Fig. 5: LegoMap as multidimensional floorplanning.

The LegoMap framework can also be interpreted as a multi-dimensional floorplanning problem. A 3D example is illustrated in Figure 5, where each color corresponds to an operator. Each block in the left part of the figure represents a candidate operator mapping solution. Its three dimensions correspond to key characteristics of that mapping solution—for example, execution time, the number of AIE cores used, and the number of PLIOs consumed. Note that a single operator may have multiple mapping options, which is why blocks of the same color appear more than once in Figure 5. During layer-level optimization, one block is selected for

each color and the selected blocks are packed in the 3D space as shown on the right side of the figure. No two blocks may overlap in this packing. Precedence constraints among operators are enforced by their order along the time axis. From the packed configuration, the overall execution time and hardware resource utilization can be determined for a layer-level solution.

C. Candidate Operator Mapping Generation

The operators considered here are primarily matrix multiplications, as they dominate computation in each transformer layer. Softmax operations are typically offloaded to the PL rather than executed on AI engines and are therefore excluded. During candidate generation, each distinct operator is defined by the dimensions of its operand matrices. If two matrix multiplications within a transformer layer share the same operand dimensions, they are treated as the same operator. Fused operators are also included in our formulation. Note that fusion is only allowed between two adjacent operators in the input DAG.

An operator mapping specifies decisions such as tiling factors, loop tile ordering, the dimensions of the AIE subarray allocated to the operator, the buffers and PLIOs assigned to it, and whether circuit or packet switching is used. Each mapping solution is evaluated in terms of execution time and hardware utilization, including AIE cores, PLIOs, and buffer usage. These metrics are estimated using the analytical models described in Section III.

For each operator, candidate mapping solutions are automatically generated and pruned in two phases. Phase I: Any candidate that is worse than another in all metrics—execution time, AIE core usage, PLIO usage, and buffer utilization—is discarded. Phase II: The remaining candidates are grouped based on similar AIE core utilization. Within each group, only the top percentile of candidates with the shortest execution times are retained.

Execution time varies significantly across different candidates, and those with short execution times are typically sparse. Consequently, Phase II is critical, as it significantly reduces the number of candidates while preserving the most promising ones. In practice, this pruning step reduces the number of candidates per operator to only a few dozen. The pruning greatly speeds up the layer-level optimization without incurring a significant sacrifice in overall solution quality.

D. Layer-Level Optimization

Given a transformer layer represented by a Directed Acyclic Graph (G), this step selects a single mapping from the candidate set for each matrix multiplication operator in G. Simultaneously, it determines the start time for every operator in G. The decisions are to minimize the overall execution time of this layer subject to hardware resource constraints on AIE cores, PLIOs and buffer. This problem is formulated and solved as ILP (Integer Linear Programming).

For each operator $v_i \in G$, a binary decision variable $x_{i,j}$ indicates whether operator v_i is realized the jth mapping among its candidates. For each mapping, we define associated parameters: execution time ($\mathrm{time}_{i,j}$), number of AIE cores required ($\mathrm{aie}_{i,j}$), number of PLIOs required ($\mathrm{plio}_{i,j}$), and buffer requirement ($\mathrm{buffer}_{i,j}$). The execution time and resource demands of operator i are computed as follows:

$$\forall r \in \{\mathrm{time}, \mathrm{aie}, \mathrm{plio}, \mathrm{buffer}\}: \quad r_i = \sum_j x_{i,j}\, r_{i,j}. \tag{12}$$

Additionally, we define timing variables T_i^{start} and T_i^{finish} to represent the start and finish times of operator i.

$$T_i^{finish} = T_i^{start} + \mathrm{time}_i \tag{13}$$

The ILP is subject to several constraints:

- Mapping selection: Each operator must be mapped to exactly one mapping:

$$\sum_j x_{i,j} = 1, \forall i. \tag{14}$$

- Resource constraints: At any time slot t, the total demand of AIE cores, PLIOs, and buffer across all active operators cannot exceed platform limits:

$$\forall r \in \{\mathrm{aie}, \mathrm{plio}, \mathrm{buffer}\}: \quad \sum_i r_i(t) \le r_{\max}. \tag{15}$$

- Precedence constraints: For each dependency $(i \to j)$ in the DAG G, the start time of operator j must be no earlier than the finish time of operator i:

$$T_j^{start} \ge T_i^{finish}, \forall (i \to j) \in G. \tag{16}$$

The objective of the problem is to minimize the overall execution time of the transformer layer, which corresponds to the finish time of the last scheduled operator:

$$\min T_{total} = \max_i \{T_i^{finish}\}. \tag{17}$$

E. ILP Acceleration

To further reduce the exploration space and accelerate the ILP optimization, we apply the following strategies.

Mapping sharing across heads and batches: We assume that operators with identical dimensions across different attention heads or inference batches share the same mapping. This assumption significantly reduces the number of binary decision variables $x_{i,j}$ in the ILP formulation, thereby simplifying the optimization problem and enabling faster solution search. While this may limit the flexibility of per-head specialization, in practice, the impact on solution quality is negligible. We further validate this assumption in the following section, where we compare shared versus specialized mappings.

Execution time upper bound pruning: Before solving the ILP, we estimate an upper bound on the total execution time, and use it to restrict the feasible range of the time variable T_i^{finish}. The upper bound is obtained by sequentially mapping all operators onto the entire AIE array, where the total execution time in this mapping is simply the sum of the execution time of the individual operators. This estimation

TABLE I: Structure of transformer layers in the experiments. Ops denotes the number of MAC operations in one layer.

Workload	Seq Length	Hidden Size	Head Size	Ops (10^9)
BERT Small	512	512	64	3.7
BERT Base	512	768	64	8.1
BERT Large	512	1024	64	13.9
ViT	256	768	64	4.8
Llama3 8B	2048	4096	128	89.3

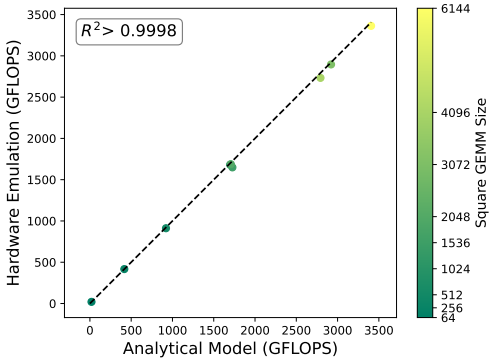

Fig. 6: Performance comparison (GFLOPs) of the analytical model and hardware emulation under different matrix sizes.

brings negligible overhead and provides an effective bound that reduces unnecessary exploration of the solution space.

These strategies effectively reduce the ILP solve time while maintaining the same solution quality in scheduling results.

V. Experiment Results

A. Experimental Setup

In the experiments, we adopted transformer layers from different models, including BERT [3], Vision Transformers [23], [24], and Llama3 8B [25]. The structures of these models and their number of operations (Ops) are listed in Table I. Our LegoMap approach is developed using C++, and we utilize Gurobi [26] as the ILP solver. The designs are compiled with the AMD/Xilinx Vitis Toolchain. The performance, e.g., throughput, of the final results is obtained from Vitis hardware emulation. Hardware resource utilization of the final results is from the Vitis reports. The hardware platform adopted in the experiments is AMD Versal Series VC1902.

In the experiments, we compare with CHARM [6]–[8], MaxEVA [9], and RSN [10]. The experiments are conducted with the 32-bit floating-point data type. Please note that CHARM is an automatic approach like LegoMap, while MaxEVA and RSN are manual design techniques. The implementation codes of CHARM and MaxEVA are obtained from GitHub repositories [27], [28], while the results of RSN are obtained from the published paper.

B. Accuracy of Analytical Performance Models

To validate the accuracy of the analytical models described in Section III, we estimate the throughput of various transformer layers using the analytical model and compare the throughput with that obtained from hardware emulation. Figure 6 shows the throughput (in GFLOPs) of various GEMMs from both our model estimation and the hardware emulation. The analytical model exhibits excellent agreement with the hardware emulation results, achieving a correlation of $R^2 > 0.9998$. The average percentage error is less than 2%.

C. Main Results

Figure 7 shows the throughput of the BERT designs from LegoMap, CHARM, and MaxEVA. One can see that our LegoMap consistently generates designs with higher throughput, compared to CHARM and MaxEVA. Another trend is that with the model size increasing, the overall throughput also increases. This throughput improvement is generally due to the higher AIE utilization with the increased operand size. This trend can also be verified from Figure 6, where a larger GEMM size gives higher throughput. In general, LegoMap demonstrates higher throughput ranging from $1.59\times - 3.33\times$ compared to CHARM and MaxEVA.

TABLE II: Resource utilization comparison across LegoMap, CHARM [6], and MaxEVA [9] on BERT Base.

Resource	LegoMap	CHARM	MaxEVA
AIE	384	288	390
PLIO Channel	175	80	154
Buffer	3014	3086	3138
CPU Time (s)	263	132	N/A

Table II demonstrates the hardware resource usage of the BERT base design across different methods. In general, LegoMap and MaxEVA utilize more AIE cores and PLIOs, bringing more computation capacity, while MaxEVA doesn't demonstrate higher throughput due to its inflexibility. Although LegoMap uses the maximum number of PLIOs, it is still less than the available PLIOs. For the CPU time, LegoMap takes longer to get a result compared to CHARM, but still remains in the same order of magnitude, which is a reasonable trade-off for better design throughput. The CPU time of MaxEVA is omitted since we obtained its design from GitHub.

TABLE III: Throughput (GFLOPs) comparison on BERT.

Workload	Batch Size	LegoMap	RSN [10]	LegoMap / RSN
BERT Large	1	1732	1618	1.07
BERT Large	3	2226	2024	1.10
BERT Large	6	2309	2138	1.08

Table III shows the throughput comparison of our framework (LegoMap) with RSN [10]. Since the paper lacks enough details to reproduce the experiment, the numbers displayed here are obtained from the paper, where only the results from BERT Large are available. On average, LegoMap achieves $1.09\times$ performance compared to RSN. Although the RSN performance results are not far from LegoMap, it is a manual design technique, while LegoMap is an automated approach.

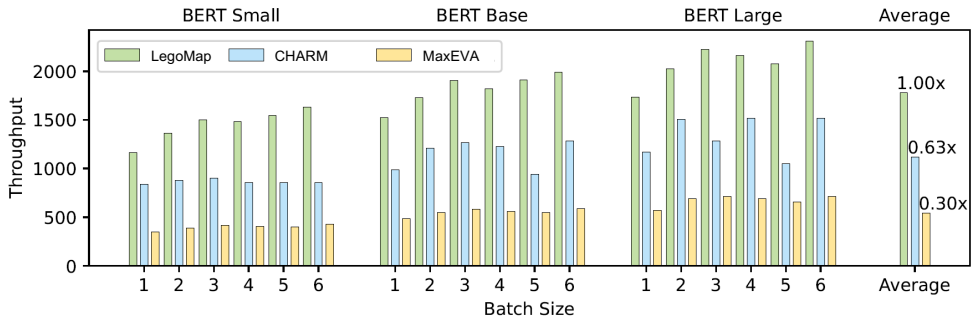

Fig. 7: Throughput (GFLOPs) comparison among LegoMap, CHARM [6], and MaxEVA [9] for various BERT models.

TABLE IV: Throughput (GFLOPs) comparison on ViT.

Batch Size	LegoMap	CHARM (norm.)	MaxEVA (norm.)
1	349.3	211.5 (0.61)	154.1 (0.44)
2	530.5	330.4 (0.62)	237.5 (0.45)
3	665.9	450.1 (0.68)	289.8 (0.44)
4	648.1	540.5 (0.83)	325.7 (0.50)
5	711.8	615.6 (0.86)	358.0 (0.50)
6	761.7	533.6 (0.70)	394.5 (0.52)
Norm. Avg	1.00	0.72	0.48

Table IV shows the throughput of the Vision Transformer designs from LegoMap, CHARM, and MaxEVA. Similar to the BERT designs, we observe a trend of increasing throughput as the batch size increases. Overall, LegoMap achieves $1.16\times$– $1.64\times$ the throughput of CHARM, and $1.9\times$–$2.3\times$ the throughput of MaxEVA.

TABLE V: Throughput (GFLOPs) comparison on Llama.

Batch	LegoMap	CHARM (norm.)	MaxEVA (norm.)
1	3695.7	2486.5 (0.67)	1556.0 (0.42)
2	3731.6	2482.7 (0.67)	1555.2 (0.42)
3	3739.0	3145.6 (0.84)	1555.0 (0.42)
4	3720.2	3066.3 (0.82)	1554.9 (0.42)
5	3723.9	3020.6 (0.81)	1554.8 (0.42)
6	3649.2	2827.1 (0.78)	1554.7 (0.43)
Norm. Avg	1.0	0.77	0.42

Table V shows the throughput comparison of Llama model across different frameworks. Since the model is too large for hardware emulation, the numbers in the table are estimated from the analytical performance models. On average, LegoMap generates designs with $1.30\times$ and $2.38\times$ higher throughput, compared with CHARM and MaxEVA.

D. Effect of ILP Acceleration Techniques

Figure 8 (a) shows the CPU time of our framework and the model execution time for the BERT small and base models, with the acceleration techniques mentioned in Section IV-E enabled or disabled. For BERT small and base models, our acceleration techniques demonstrate $8.7\times$ and $12.9\times$ speedup on the CPU time respectively. For BERT base model with acceleration techniques disabled, the ILP solver failed to find

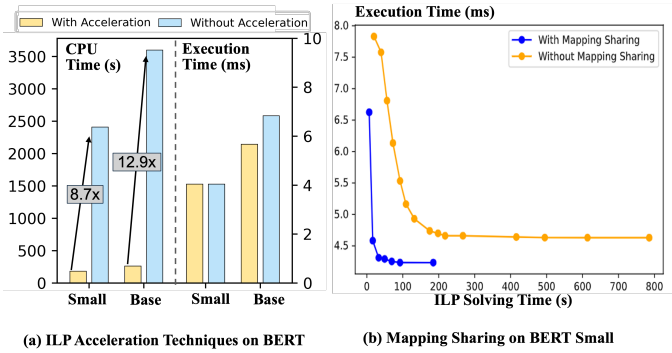

Fig. 8: (a) CPU Time and model execution time of BERT Small/Base models with and without acceleration techniques. (*ILP on BERT base times out due to time limit.) (b) Model execution time versus ILP solving time in ILP optimization on the BERT small model.

the minimum solution within the time bound, thus giving a worse model execution time than the default setting. Also, from the execution time comparison, one can see that our acceleration technique doesn't sacrifice the solution quality.

Figure 8 (b) shows the process of the ILP solver optimizing the model execution time for the BERT small model. The blue curve shows the process with "Mapping Sharing" enabled, while the orange curve shows the one disabled. One can see that in the blue curve, the objective function value (in layer execution time) drops quickly and reaches the minimum value within several minutes. While in the orange curve, ILP struggles to reach the minimum. This comparison demonstrates the effect of Mapping Sharing during the ILP in layer-level optimization.

VI. Conclusion

We presented an automatic framework for mapping transformer layers onto the Versal ACAP platform. Our framework LegoMap jointly optimizes hardware resource partitioning and operator scheduling using ILP, and applies design-space reduction strategies to make exploration tractable. Experiments on various transformer layers demonstrate up to 30%-58% throughput improvement over the state-of-the-art automatic approach, showing the effectiveness of our approach in accelerating transformer computing on AI engine-based FPGAs.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[2] A. Conneau and G. Lample, "Cross-lingual language model pretraining," in *Advances in Neural Information Processing Systems*, 2019.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019.

[4] A. Kolesnikov, A. Dosovitskiy, D. Weissenborn, G. Heigold, J. Uszkoreit, L. Beyer, M. Minderer, M. Dehghani, N. Houlsby, S. Gelly, T. Unterthiner, and X. Zhai, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[5] B. Gaide, D. Gaitonde, C. Ravishankar, and T. Bauer, "Xilinx adaptive compute acceleration platform: Versaltm architecture," in *International Symposium on Field-Programmable Gate Arrays*, 2019.

[6] J. Zhuang, J. Lau, H. Ye, Z. Yang, Y. Du, J. Lo, K. Denolf, S. Neuendorffer, A. Jones, J. Hu, D. Chen, J. Cong, and P. Zhou, "Charm: Composing heterogeneous accelerators for matrix multiply on versal acap architecture," in *International Symposium on Field Programmable Gate Arrays*, 2023.

[7] J. Zhuang, J. Lau, H. Ye, Z. Yang, S. Ji, J. Lo, K. Denolf, S. Neuendorffer, A. Jones, J. Hu, Y. Shi, D. Chen, J. Cong, and P. Zhou, "Charm 2.0: Composing heterogeneous accelerators for deep learning on versal acap architecture," *ACM Transactions on Reconfigurable Technology and Systems*, 2024.

[8] J. Zhuang, Z. Yang, and P. Zhou, "High performance, low power matrix multiply design on acap: from architecture, design challenges and dse perspectives," in *Design Automation Conference*, 2023.

[9] E. Taka, A. Arora, K.-C. Wu, and D. Marculescu, "Maxeva: Maximizing the efficiency of matrix multiplication on versal ai engine," in *International Conference on Field Programmable Technology*, 2023.

[10] C. Wang, X. Zhang, J. Cong, and J. C. Hoe, "Reconfigurable stream network architecture," in *International Symposium on Computer Architecture*, 2025.

[11] P. Dong, J. Zhuang, Z. Yang, S. Ji, Y. Li, D. Xu, H. Huang, J. Hu, A. K. Jones, Y. Shi, Y. Wang, and P. Zhou, "Eq-vit: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on versal acap architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[12] J. Zhuang, Z. Yang, S. Ji, H. Huang, A. K. Jones, J. Hu, Y. Shi, and P. Zhou, "Ssr: Spatial sequential hybrid architecture for latency throughput tradeoff in transformer acceleration," in *International Symposium on Field Programmable Gate Arrays*, 2024.

[13] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Compute and energy consumption trends in deep learning inference," *arXiv preprint arXiv:2109.05472*, 2021.

[14] F. D. Keles, P. M. Wijewardena, and C. Hegde, "On the computational complexity of self-attention," in *International Conference on Algorithmic Learning Theory*, 2023.

[15] Z. Shen, M. Zhang, H. Zhao, S. Yi, and H. Li, "Efficient attention: Attention with linear complexities," in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision*, 2021.

[16] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "Gobo: Quantizing attention-based NLP models for low latency and energy efficient inference," in *International Symposium on Microarchitecture*, 2020.

[17] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022.

[18] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[19] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[20] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "FLAT: an optimized dataflow for mitigating attention bottlenecks," in *International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023.

[21] S. Zheng, S. Chen, S. Gao, L. Jia, G. Sun, R. Wang, and Y. Liang, "Tileflow: A framework for modeling fusion dataflow via tree-based analysis," in *International Symposium on Microarchitecture*, 2023.

[22] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, "Chimera: An analytical optimizing framework for effective compute-intensive operators fusion," in *International Symposium on High-Performance Computer Architecture*, 2023.

[23] B. Wu, C. Xu, X. Dai, A. Wan, P. Zhang, Z. Yan, M. Tomizuka, J. Gonzalez, K. Keutzer, and P. Vajda, "Visual transformers: Token-based image representation and processing for computer vision," *arXiv preprint arXiv:2006.03677*, 2020.

[24] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Conference on Computer Vision and Pattern Recognition*, 2009.

[25] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[26] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2024. [Online]. Available: https://www.gurobi.com

[27] J. Zhuang, J. Lau, H. Ye, Z. Yang, Y. Du, J. Lo, K. Denolf, S. Neuendorffer, A. Jones, J. Hu, D. Chen, J. Cong, and P. Zhou, "Charm: Composing heterogeneous accelerators for matrix multiply on versal acap architecture," https://github.com/arc-research-lab/CHARM, 2023.

[28] E. Taka, A. Arora, K.-C. Wu, and D. Marculescu, "Maxeva: Maximizing the efficiency of matrix multiplication on versal ai engine," https://github.com/enyac-group/MaxEVA, 2023.